

AGENT WORKFLOW RUNNER

Technical Reference & Architecture Guide

A step-by-step guide to how and why everything works

Kubernetes · FastAPI · Redis · LangGraph · Claude API · Prometheus · Grafana

SECTION 1 — SYSTEM OVERVIEW

What this system does

The Agent Workflow Runner is a distributed AI task platform. A client submits a task over HTTP. The system queues it, provisions an isolated Kubernetes Job, runs a Claude-powered agent inside that Job, and makes the result available for polling. Every step is observable via Prometheus metrics and Grafana dashboards.

WHY

Most AI agent demos run inside a single process — one crashed agent takes down the entire server, one slow task blocks others, and there is no way to observe what is happening. This project treats AI workloads the way production MLOps teams do: as isolated, schedulable, observable units of compute.

Full system architecture

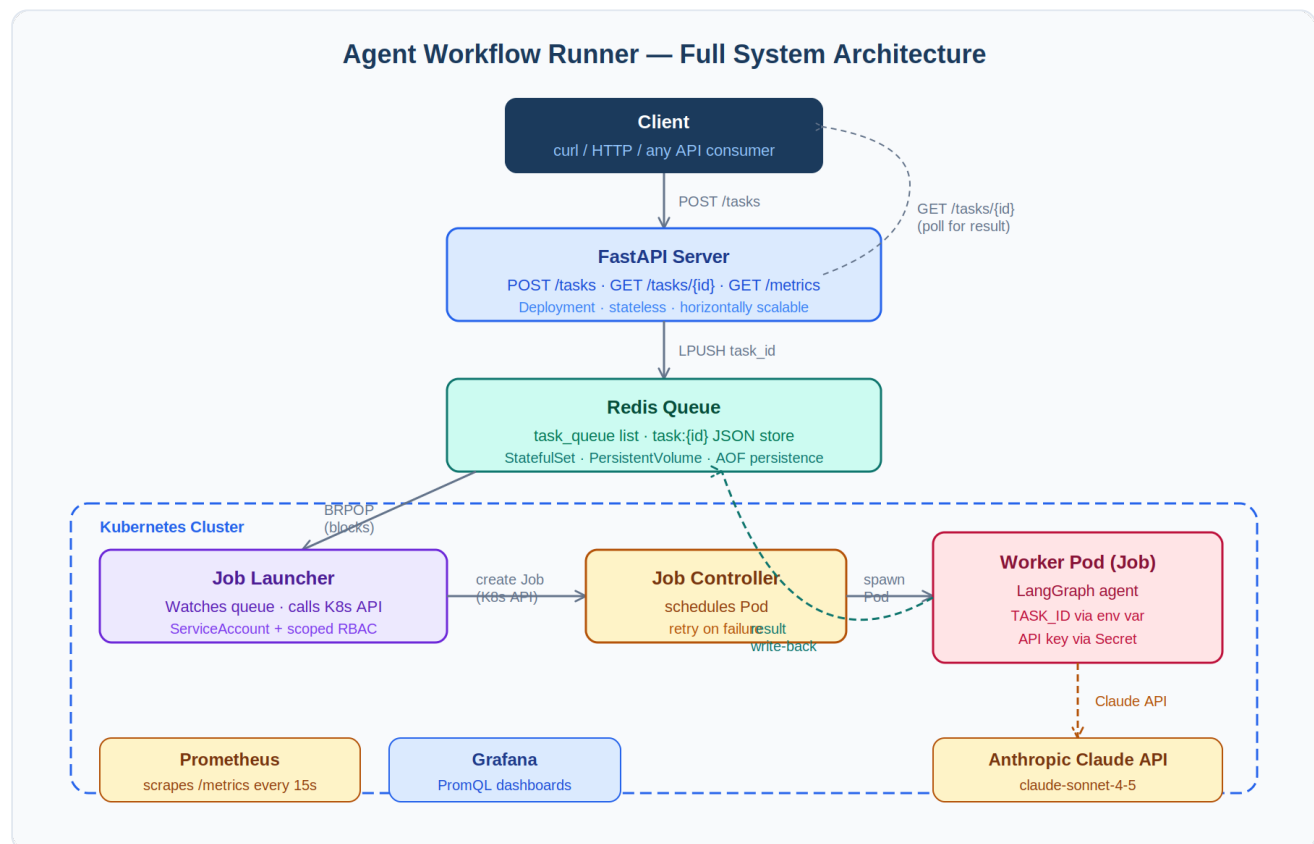


Figure 1 — Full system architecture. Client → FastAPI → Redis → Job Launcher → K8s Job → Worker Pod → Claude API.

The data flow — end to end

Every task follows this exact path through the system:

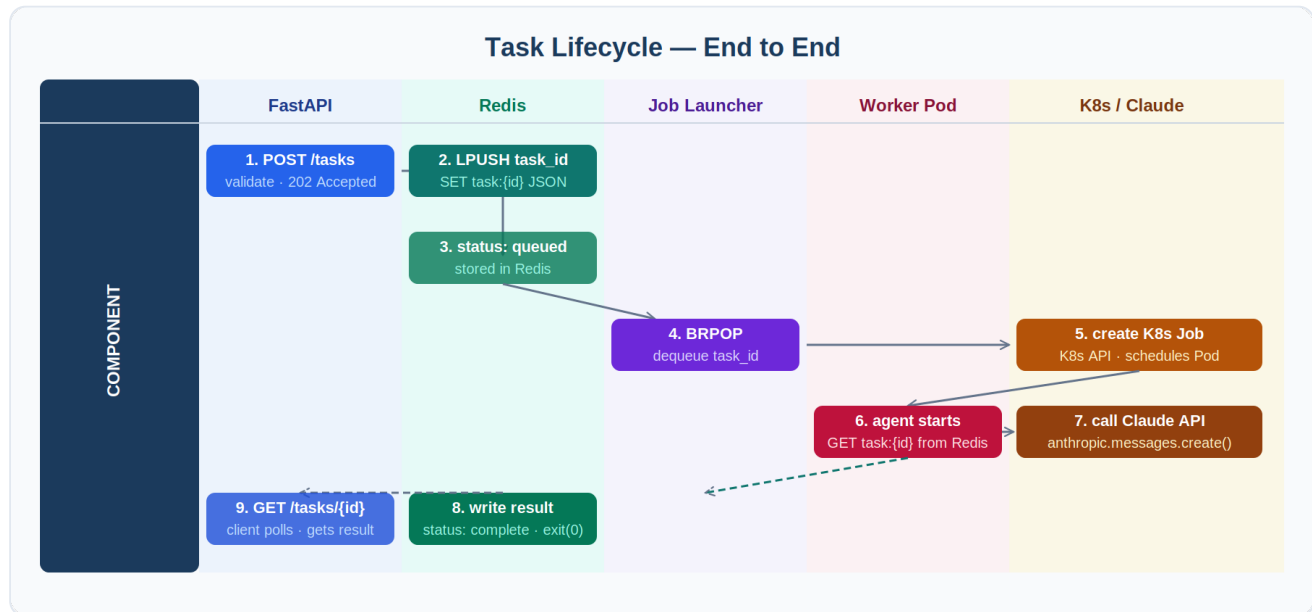


Figure 2 — Task lifecycle swimlane. Each column is one component; each row is one step in the flow.

1

Client submits POST /tasks

HTTP POST with type and input. FastAPI validates the payload and returns 202 Accepted in under 10ms — never waits for the agent to run.

2

FastAPI enqueues to Redis

Writes full task JSON to `task:{id}` in Redis. Pushes `task_id` onto `task_queue` list with LPUSH. Two atomic operations.

3

Job Launcher dequeues

BRPOP blocks the launcher thread with zero CPU overhead until a `task_id` arrives. Atomically dequeues — only one launcher ever gets each task.

4

Launcher creates K8s Job

K8s Python client builds a V1Job manifest in code — same as writing YAML — and posts it to the K8s API server via the ServiceAccount token.

5

K8s schedules a Pod

The Job controller creates a Pod on an available agent node. Injects `TASK_ID` env var and mounts the `ANTHROPIC_API_KEY` Secret.

6

Agent runs and calls Claude

agent.py fetches the task from Redis, sets status to `running`, calls the Anthropic API with a system prompt shaped by task type.

7

Worker writes result, exits 0

Result stored in `task:{id}`. Worker calls FastAPI to trigger metric recording. Exits with code 0 — Job controller marks Job Complete.

8

Client polls GET /tasks/{id}

FastAPI reads task JSON from Redis. Status field shows `complete`. Result field contains the Claude response.

Kubernetes primitives used

Deployment vs StatefulSet vs Job

This is the most important conceptual distinction in the project. Three workload types are used, each chosen deliberately.

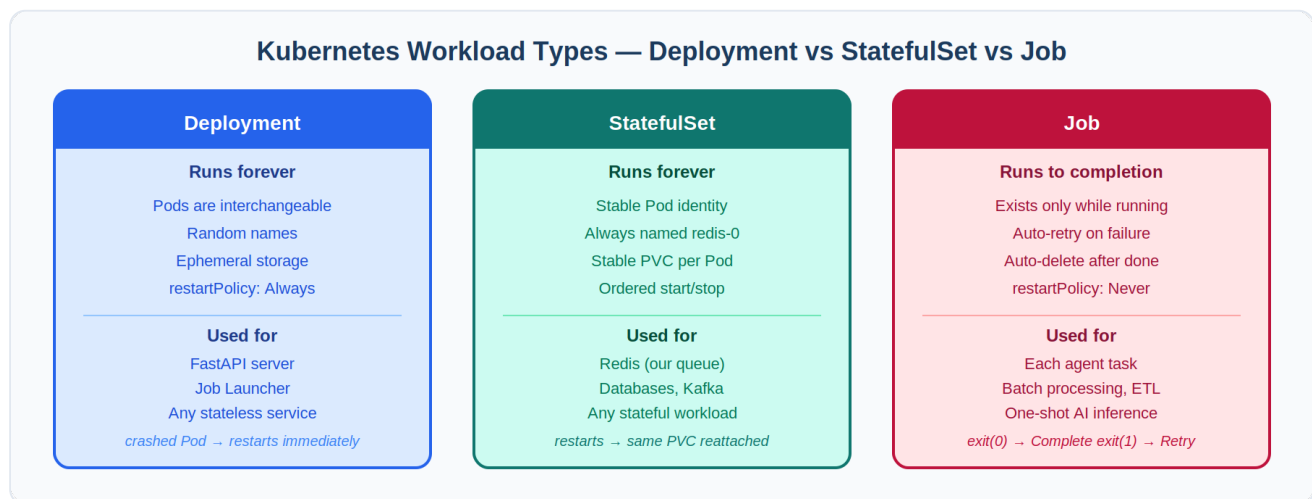


Figure 3 — The three Kubernetes workload types. `restartPolicy` is the field that defines the difference.

	Deployment	StatefulSet	Job
Lifecycle	Runs forever	Runs forever	Runs to completion
Pod identity	Random, replaceable	Stable (redis-0)	Exists while running
Storage	Ephemeral	Stable PVC per Pod	Ephemeral
Success condition	Pods stay running	Pods stay running	Pod exits 0
restartPolicy	Always	Always	Never
Used for	FastAPI, Launcher	Redis	Each agent task

KEY INSIGHT

restartPolicy: Never is what makes a Pod a Job. With Deployments, a crashed Pod restarts place. With Jobs, the dead Pod is left for log inspection and the Job controller decides whether to create a fresh Pod for a retry. Failed Job Pod logs survive even after retry has started — critical for debugging.

Persistent storage — PV, PVC, StorageClass

Redis needs its data to survive Pod restarts. Kubernetes uses a three-layer storage abstraction that separates *what an app wants* from *where storage actually comes from*.

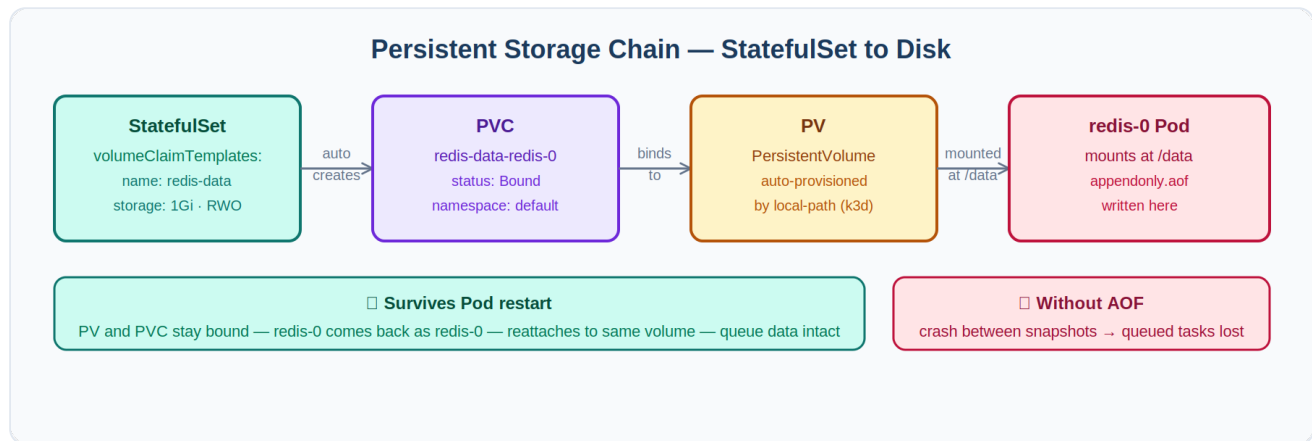


Figure 4 — The PV/PVC/StorageClass chain. StatefulSet declares a template; K8s provisions the rest automatically.

Object	What it is	Who creates it
PersistentVolume (PV)	The actual storage — a directory on a node (k3d), an EBS volume (AWS), a persistent disk (GCP). Cluster-scoped, infrastructure-level.	StorageClass provisioner — automatically on k3d and cloud clusters. Never write PV manifests yourself.
PersistentVolumeClaim (PVC)	Your app's request for storage: I need 1Gi, ReadWriteOnce. Namespace-scoped, application-level. One-to-one binding with a PV is permanent.	StatefulSet creates it automatically from <code>volumeClaimTemplates</code> . Named <code>redis-data-redis-0</code> .
StorageClass	Defines how PVs get provisioned. k3d uses <code>local-path</code> . EKS uses <code>gp3</code> . GKE uses <code>standard</code> . Your manifests only reference PVCs — StorageClass is transparent.	Pre-installed by the cluster provider. You reference it by name.

The full chain for Redis:

```
StatefulSet volumeClaimTemplates declares: name=redis-data, 1Gi, ReadWriteOnce
→ K8s auto-creates PVC: redis-data-redis-0
→ local-path StorageClass provisions a directory on the k3d node
→ PV is created and bound to the PVC (status: Bound)
→ redis-0 Pod mounts the PVC at /data
→ Redis writes appendonly.aof to /data
→ Pod restarts → redis-0 reattaches to same PVC → data intact
```

PRODUCTION NOTE

The `reclaimPolicy` on the StorageClass controls what happens to the PV when the PVC is deleted. **Delete** (cloud default) destroys the underlying disk — data gone. **Retain** keeps the PV for manual recovery. Use Delete for a job queue. Use Retain for a primary database so a mistaken `kubectl delete pvc` does not destroy production data.

RBAC — identity, permissions, binding

The job launcher calls the Kubernetes API to create Jobs. RBAC controls exactly what it is permitted to do — and nothing more.

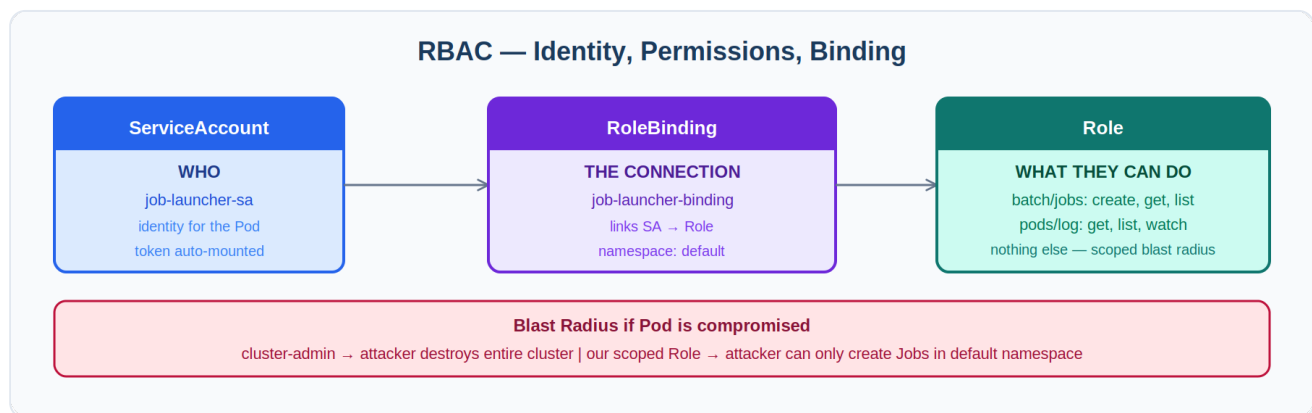


Figure 5 — RBAC three-object pattern. ServiceAccount = who. Role = what they can do. RoleBinding = the connection.

Object	Role	Our configuration
ServiceAccount	The identity the Pod runs as. K8s mounts a token at <code>/var/run/secrets/kubernetes.io/serviceaccount/token</code> automatically. The K8s Python client reads this token — no passwords or kubeconfig files needed.	<code>job-launcher-sa</code> in the default namespace
Role	Defines what API calls are allowed: which apiGroups, resources, and verbs. This is the principle of least privilege — only grant what is absolutely required.	create/get/list/watch/delete on <code>batch/jobs</code> and get/list/watch on <code>pods/log</code> . Nothing else.
RoleBinding	The object that connects the ServiceAccount to the Role. Without a RoleBinding the ServiceAccount has zero permissions even if the Role exists.	<code>job-launcher-binding</code> links SA → Role in default namespace

BLAST RADIUS

If the job-launcher Pod is compromised — malicious agent output, supply chain attack — an attack with cluster-admin credentials can destroy your entire cluster. An attacker limited by our scoped Role can only create Jobs in the default namespace. That difference is why RBAC scoping is non-negotiable in production.

SECTION 3 — COMPONENTS IN DEPTH

Each component explained

FastAPI — the front door

FastAPI is deliberately kept narrow. It has exactly three responsibilities:

- Accept task submissions — validate payload, write to Redis, return 202 Accepted in under 10ms
- Answer status checks — read `task:{id}` from Redis and return the full JSON
- Expose metrics — Prometheus scrapes `/metrics` every 15 seconds via ServiceMonitor

FastAPI knows nothing about LangGraph, Claude, or Kubernetes. It is purely an HTTP interface to Redis. This keeps it stateless, horizontally scalable, and independently deployable.

ASYNC PATTERN

`POST /tasks` returns 202 Accepted immediately — never waits for the agent. This is the async task pattern, the most important architectural idea in distributed backend systems. The naive alternative (running the agent inline) breaks on HTTP timeouts, wastes threads, and couples API latency directly to Claude API latency.

Key endpoints

<code>POST /tasks</code>	→ 202 Accepted + { task_id, status: "queued" }
<code>GET /tasks/{task_id}</code>	→ { task_id, status, result, duration_seconds }
<code>GET /healthz</code>	→ { status: "ok", redis: "connected" } (readiness probe)
<code>GET /metrics</code>	→ Prometheus exposition format (scraped every 15s)

Redis — the job queue

Redis is the backbone of the system. It holds two distinct data structures:

<code>task:{id}</code>	→ JSON string: full task state including status, input, result, timestamps
<code>task_queue</code>	→ Redis List: FIFO queue of task_ids. LPUSH to enqueue, BRPOP to dequeue.

Why Redis, not Postgres

BRPOP (Blocking Right Pop) is the key primitive. A worker calls `BRPOP task_queue 0` and the OS puts the thread to sleep with zero CPU overhead until an item arrives. The moment FastAPI calls LPUSH, Redis atomically wakes exactly one waiting worker. No polling, no locking, no coordination code.

A Postgres-based queue needs: a status column, `SELECT FOR UPDATE SKIP LOCKED`, polling on an interval, a reaper job for tasks stuck in IN_PROGRESS after a crash. That is roughly 200 lines of infrastructure code to approximate two Redis commands.

Persistence — why AOF matters

```
redis-server --appendonly yes --appendfsync everysec
```

AOF (Append-Only File) logs every write operation to `/data/appendonly.aof`. On restart, Redis replays the log and recovers all state. At most 1 second of data loss on a crash. Without AOF, Redis only takes periodic snapshots — a crash between snapshots silently drops all queued tasks.

Job Launcher — the orchestrator

The job launcher bridges Redis and the Kubernetes API. It is the most Kubernetes-native piece of code in the project.

Function	What it does
<code>load_k8s_config()</code>	Inside a cluster: reads ServiceAccount token from the auto-mounted path. Outside: falls back to <code>~/.kube/config</code> . No manual credential management needed.
BRPOP loop	Blocks on <code>task_queue</code> with a 30-second timeout. Zero CPU overhead while idle. Timeout wakes the loop every 30s as a health check then blocks again.
<code>build_job_manifest()</code>	Constructs a V1Job Python object — identical to writing a Job YAML manifest but in code. Sets <code>backoffLimit=2</code> , <code>activeDeadlineSeconds=300</code> , <code>ttlSecondsAfterFinished=600</code> .
<code>create_namespaced_job()</code>	Posts the Job to the K8s API server via BatchV1Api. The Job controller takes over: schedules the Pod, watches completion, handles retries.

OPERATOR PATTERN

The launcher is a minimal implementation of the Kubernetes operator pattern — a process that watches external state and reconciles K8s resources to match it. Argo Workflows, Kubeflow Pipelines, and Apache Airflow's KubernetesExecutor all work on this same model internally.

Worker Agent — runs inside each Job Pod

agent.py is a run-to-completion script. It starts, does one unit of work, and exits. No server, no long-running loop. The Pod lifecycle maps directly to the task lifecycle.

```
1. Read env vars      TASK_ID, REDIS_HOST, ANTHROPIC_API_KEY — all from Job manifest
2. Fetch task         GET task:{TASK_ID} from Redis — gets type and input
3. Update status      SET task:{TASK_ID} status=running — polling clients see progress
4. Start timer        time.time() — measures actual agent duration
5. Call Claude        client.messages.create() with system prompt by task type
6. Write result       SET task:{TASK_ID} with result + duration_seconds
7. Notify FastAPI     GET /tasks/{id} — triggers metric recording
8. exit(0) or exit(1) 0 = Job Complete · 1 = retry up to backoffLimit then Failed
```

The exit code is the only signal the Job controller sees. Exit 0 means success. Anything else means failure. K8s respects it. Your retry logic is free.

SECTION 4 — OBSERVABILITY

Prometheus and Grafana

Observability is how you know the system is healthy without staring at individual logs. The Prometheus + Grafana stack gives you time-series metrics, alerting, and visual dashboards.

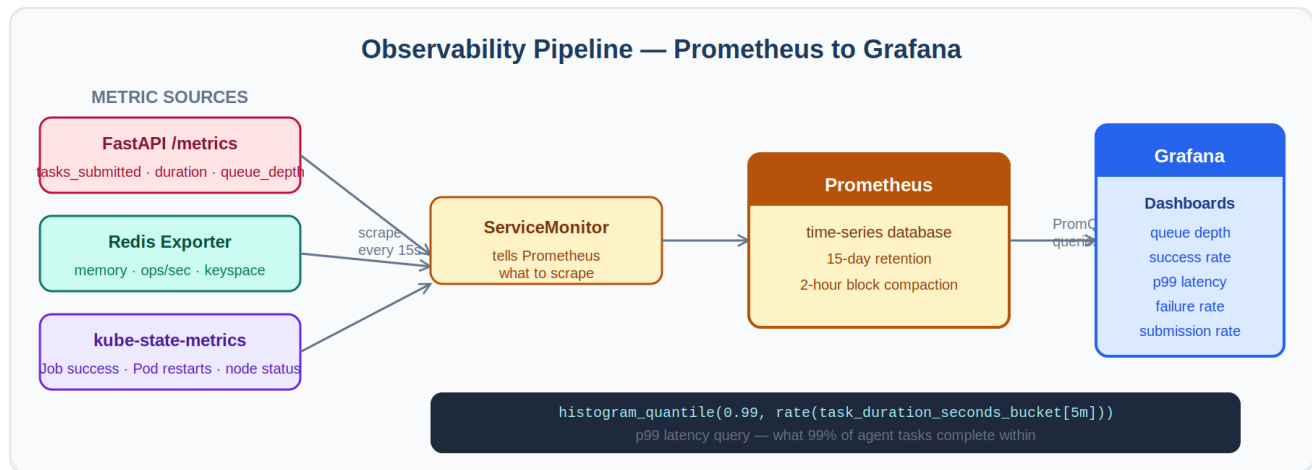


Figure 6 — Observability pipeline. Metric sources → ServiceMonitor → Prometheus scrape → PromQL → Grafana.

The three Prometheus metric types

Type	When to use it	Our usage
Counter	Only ever goes up. Use for cumulative events: requests, errors, completions. Never for values that decrease.	<code>tasks_submitted_total</code> , <code>tasks_completed_total</code>
Histogram	Measures distribution of values in buckets. Use for latency, duration, size. Enables p50/p95/p99 via <code>histogram_quantile()</code> . Captures the slow tail that averages hide.	<code>task_duration_seconds_bucket</code>
Gauge	Goes up and down. Use for current state: queue depth, active connections, memory. Unlike a Counter it can decrease.	<code>task_queue_depth</code>

PromQL — the four patterns you need

PromQL is not SQL. There are no joins or rows. Every expression transforms time series. These four patterns cover 90% of real-world usage.

1. Raw instant vector

```
tasks_submitted_total
```

Returns the current value of every matching series. Useful for Gauges. For Counters the raw value is cumulative since startup — use `rate()` instead.

2. `rate()` over a range vector

```
rate(tasks_submitted_total[5m])
```

Per-second rate of increase averaged over 5 minutes. The primary pattern for Counters. Window tradeoff: `[1m]` is spiky and reactive, `[15m]` is smooth but slow. `[5m]` is the standard starting point.

3. Label filtering

```
rate(tasks_completed_total{status="failed"}[5m])
```

Filter to specific series by label value. `=` for exact match, `!=` for exclusion, `=~` for regex. Combine filters with commas. Labels are how you split metrics by task type, status, environment, or any dimension.

4. `histogram_quantile()` for percentiles

```
histogram_quantile(0.99, rate(task_duration_seconds_bucket[5m]))
```

Compute the 99th percentile from histogram bucket counts. P99 means 99% of tasks complete faster than this value. More meaningful than average — captures the worst user experiences. A fine-looking average with a P99 of 240s means you have a real problem.

How Prometheus discovers FastAPI — the ServiceMonitor chain

ServiceMonitors are custom resources that tell the Prometheus operator what to scrape. The discovery chain has two hops and both must match.

Check	What must match
ServiceMonitor labels	<code>release: prometheus-stack</code> must match the Helm release name. Without this the Prometheus operator ignores the ServiceMonitor entirely.
ServiceMonitor selector	<code>matchLabels: app: fastapi</code> must match a label on the Service object — not the Pods. This is a Service lookup.
Service port name	The <code>port</code> field in ServiceMonitor <code>endpoints</code> is a NAME lookup , not a port number. The Service must have a named port (e.g. <code>name: http</code>) that exactly matches the string.
Service selector	The Service's own <code>spec.selector</code> routes to Pods with <code>app: fastapi</code> . Prometheus follows this to find actual Pod IPs to scrape.

DEBUGGING ORDER

When metrics are missing: (1) check operator logs for `skipping object` warnings — one bad ServiceMonitor blocks all reconciliation. (2) check `http://localhost:9090/targets` for UP/DO status. (3) `curl /metrics` directly to confirm the endpoint works. (4) verify Service has the label ServiceMonitor selects on. (5) verify the `port` field is a name not a number.

SECTION 5 — GIT STRATEGY & NEXT STEPS

Pushing to GitHub in logical phases

A clean commit history tells the story of how you built the system. Each commit should represent a working, deployable state — not a mid-feature checkpoint. A reviewer should be able to check out any commit and run the system.

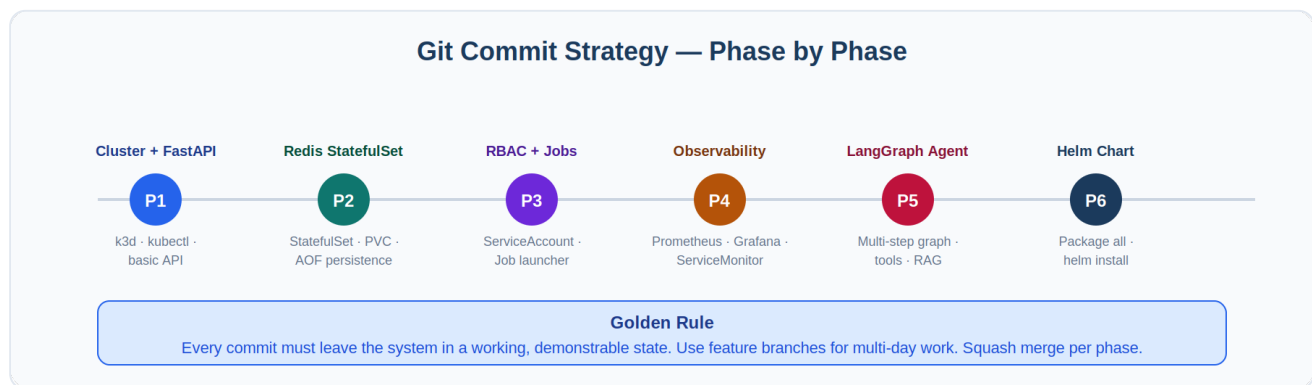


Figure 7 — Six-phase commit strategy. Each phase is a complete working system that builds on the previous one.

Phase 1	feat: k3d cluster setup and FastAPI skeleton app/main.py (in-memory dict), app/Dockerfile, k8s/fastapi-deployment.yaml, k8s/fastapi-service.yaml. Verify: kubectl get pods shows Running, curl /tasks returns 202 Accepted.
Phase 2	feat: Redis StatefulSet with persistent storage k8s/redis-statefulset.yaml, k8s/redis-service.yaml. Updated app/main.py to use Redis. Verify: tasks survive FastAPI Pod restart, redis-cli LLEN task_queue shows queued tasks.
Phase 3	feat: RBAC, secrets, Job Launcher, and worker agent k8s/rbac.yaml, k8s/secret.yaml, k8s/job-launcher-deployment.yaml, launcher.py, worker/agent.py, worker/Dockerfile. Verify: submit task, watch agent-task-* Pod appear, complete, and return result.
Phase 4	feat: Prometheus and Grafana observability k8s/fastapi-servicemonitor.yaml, k8s/redis-exporter.yaml. Updated app/main.py with Counter/Histogram/Gauge. Verify: tasks_submitted_total in Prometheus, live Grafana dashboard.

Phase 5

feat: LangGraph multi-step agent with tools

Updated worker/agent.py with LangGraph graph, web search tool, and RAG retrieval. Verify: agent runs multiple steps visible in Pod logs, steps_per_task histogram in Grafana.

Phase 6

feat: Helm chart packaging full application

charts/agent-runner/ directory with templates for all resources. Verify: helm install agent-runner ./charts/agent-runner deploys everything to a fresh namespace from scratch.

RULE

Every commit must leave the system in a working, demonstrable state. For multi-day work, use a feature branch and squash-merge when the phase is complete. This keeps the main branch clean and every commit checkout-able for a demo.

Option A — Helm chart

Package the entire application into a single installable chart. This is how every serious Kubernetes application is distributed and deployed.

Target end state

```
helm install agent-runner ./charts/agent-runner \
  --namespace agent-runner \
  --create-namespace \
  --set anthropicApiKey=$ANTHROPIC_API_KEY \
  --set workerImage.tag=v4
```

Chart structure to build

```
charts/agent-runner/
├─ Chart.yaml           # name, version, appVersion, description
├─ values.yaml          # defaults for all configurable values
├─ templates/
│  └─ _helpers.tpl      # named templates: labels, selectors, names
│  └─ fastapi-deployment.yaml
│  └─ fastapi-service.yaml
│  └─ redis-statefulset.yaml
│  └─ redis-service.yaml
│  └─ job-launcher-deployment.yaml
│  └─ rbac.yaml
│  └─ secret.yaml
│  └─ servicemonitor.yaml
└─ charts/              # sub-chart dependencies (optional)
```


Key Helm concepts to learn

- `values.yaml` — single source of truth for all configuration. Everything that changes between environments lives here.
- `{{ .Values.image.tag }}` — Go templating syntax. Values are injected into manifests at install time.
- `{{ include "agent-runner.labels" . }}` — named templates in `_helpers.tpl` ensure consistent labels across all resources.
- `helm template ./charts/agent-runner` — renders manifests locally without installing. Use this to debug template errors before deploying.
- `helm upgrade --install` — idempotent. Works whether the chart is already installed or not. Use this in CI pipelines.

Option B — LangGraph multi-step agent

Replace the single Claude call in `worker/agent.py` with a real multi-step LangGraph graph with tools. The infrastructure does not change — only the agent internals become more sophisticated.

Target graph architecture

```
from langgraph.graph import StateGraph

graph = StateGraph(AgentState)
graph.add_node('plan', plan_task) # Claude breaks task into steps
graph.add_node('research', web_search) # searches for current information
graph.add_node('reason', reason_step) # Claude reasons over results
graph.add_node('write', write_output) # Claude writes the final answer
graph.add_node('done', finalize) # writes result to Redis, exits

graph.set_entry_point('plan')
graph.add_conditional_edges('plan', route_after_plan)
graph.add_edge('research', 'reason')
graph.add_edge('reason', 'write')
graph.add_edge('write', 'done')
```

Tools to add

- Web search — Tavily API or DuckDuckGo. Gives the agent real-time information beyond its training data.
- Code execution — sandboxed Python interpreter. Agent can write and run code as part of research tasks.
- RAG retrieval — FAISS or Qdrant vector store. Maps directly to the text-to-SQL RAG work from Qualcomm.

SECTION 6 — DEBUGGING PLAYBOOK

Systematic debugging reference

The debugging sequence matters as much as individual commands. Always narrow the failure domain before trying fixes. The pattern: check events → check logs → check configuration → check connectivity.

Pod not starting

```
kubectl describe pod <pod-name> # Events section – always first
kubectl get events --sort-by=.lastTimestamp # cluster-wide event log
kubectl get pod <pod-name> -o yaml | grep -A5 containerStatuses
```

Symptom	Root cause and fix
ErrImageNeverPull	Image not imported into k3d. Run: <code>k3d image import <image>:tag -c agentcluster</code>
ImagePullBackOff	Image not found in registry. Check image name and tag spelling exactly.
Pending (no node)	Insufficient resources. Check: <code>kubectl describe node</code> and <code>kubectl top nodes</code>
CreateContainerConfigError	Referenced Secret or ConfigMap does not exist. Check names match exactly.
Unschedulable	Taints on nodes that Pod cannot tolerate. Check node taints with <code>kubectl describe node</code> .

Pod starts but crashes

```
kubectl logs <pod-name> # stdout/stderr of current container
kubectl logs <pod-name> -p # logs from previous crashed container
kubectl logs job/<job-name> # logs from the Job's Pod
kubectl describe job <job-name> # retry count, completion time, failure reason
```

Symptom in logs	Root cause and fix
ModuleNotFoundError	Package missing from requirements.txt. Add it, rebuild image, reimport to k3d.
TypeError: unexpected keyword argument	Dependency version conflict. Pin compatible versions explicitly in requirements.txt.
ConnectionRefusedError / redis	redis-service DNS not resolving. Check Service selector labels match Pod labels.
AuthenticationError / Anthropic	Secret not mounted. Check secretKeyRef name and key spelling match exactly.
BackoffLimitExceeded	Job exhausted retries. Check Pod logs from all failed attempts with <code>kubect1 logs -p</code> .

Metrics missing in Prometheus or Grafana

```
# Step 1 – check if Prometheus operator is rejecting ServiceMonitors
kubect1 logs -n monitoring deployment/prometheus-stack-kube-prom-operator --tail=50

# Step 2 – check Prometheus target list (look for UP or DOWN)
# http://localhost:9090/targets

# Step 3 – verify metrics endpoint directly
curl http://localhost:9000/metrics | grep tasks_

# Step 4 – verify reachable from inside the cluster
kubect1 run debug --image=curlimages/curl -n monitoring --restart=Never --rm -it \
  -- curl http://fastapi-service.default.svc.cluster.local:80/metrics

# Step 5 – verify Service label matches ServiceMonitor selector
kubect1 get service fastapi-service -o jsonpath='{.metadata.labels}'

# Step 6 – verify ServiceMonitor port is a NAME not a number
kubect1 get servicemonitor -n monitoring fastapi-monitor -o yaml
```

Redis inspection commands

```
kubectl exec -it redis-0 -- redis-cli
```

```
PING                # returns PONG if Redis is healthy
KEYS *              # all keys in the database
LLEN task_queue     # tasks waiting in the queue
LRANGE task_queue 0 -1 # list all queued task_ids
GET task:<id>        # full JSON for a specific task
INFO persistence    # AOF status and last write time
```

Essential kubectl reference

Command	What it tells you
<code>kubectl get all</code>	Everything in the default namespace at a glance
<code>kubectl get pods -w</code>	Watch Pod status changes in real time (Ctrl+C to stop)
<code>kubectl rollout status deployment/fastapi</code>	Watch a rolling update complete
<code>kubectl get jobs</code>	All Jobs with completion status and age
<code>kubectl delete jobs --field-selector status.successful=0</code>	Clean up all failed Jobs
<code>kubectl get pvc</code>	PersistentVolumeClaims and whether they are Bound
<code>kubectl get pv</code>	PersistentVolumes and which PVC each is bound to
<code>kubectl get storageclass</code>	Available storage classes and their provisioners
<code>kubectl exec -it <pod> -- /bin/bash</code>	Interactive shell inside a running container
<code>kubectl port-forward svc/<name> 9000:80</code>	Forward local port to a cluster Service for testing
<code>kubectl describe <kind> <name></code>	Full spec + Events — the most useful debugging command
<code>kubectl get events --sort-by=.lastTimestamp</code>	Chronological cluster event log for timing issues

SECTION 7 — LANGGRAPH MULTI-STEP AGENT

How the agent thinks

The worker Pod no longer makes a single Claude call. It runs a LangGraph state machine where Claude makes real decisions at each node — whether to search the web, what to search for, how to reason about results, and how to synthesize a final answer. The infrastructure layer is completely unchanged.

KEY SHIFT

The previous agent had no memory between steps. The LangGraph agent accumulates context as it moves through nodes — each step builds on what came before. By the time it reaches `done`, the state contains the full reasoning trace. That trace is written to Redis and returned via the API so you can inspect exactly how Claude reasoned through every task.

The graph architecture

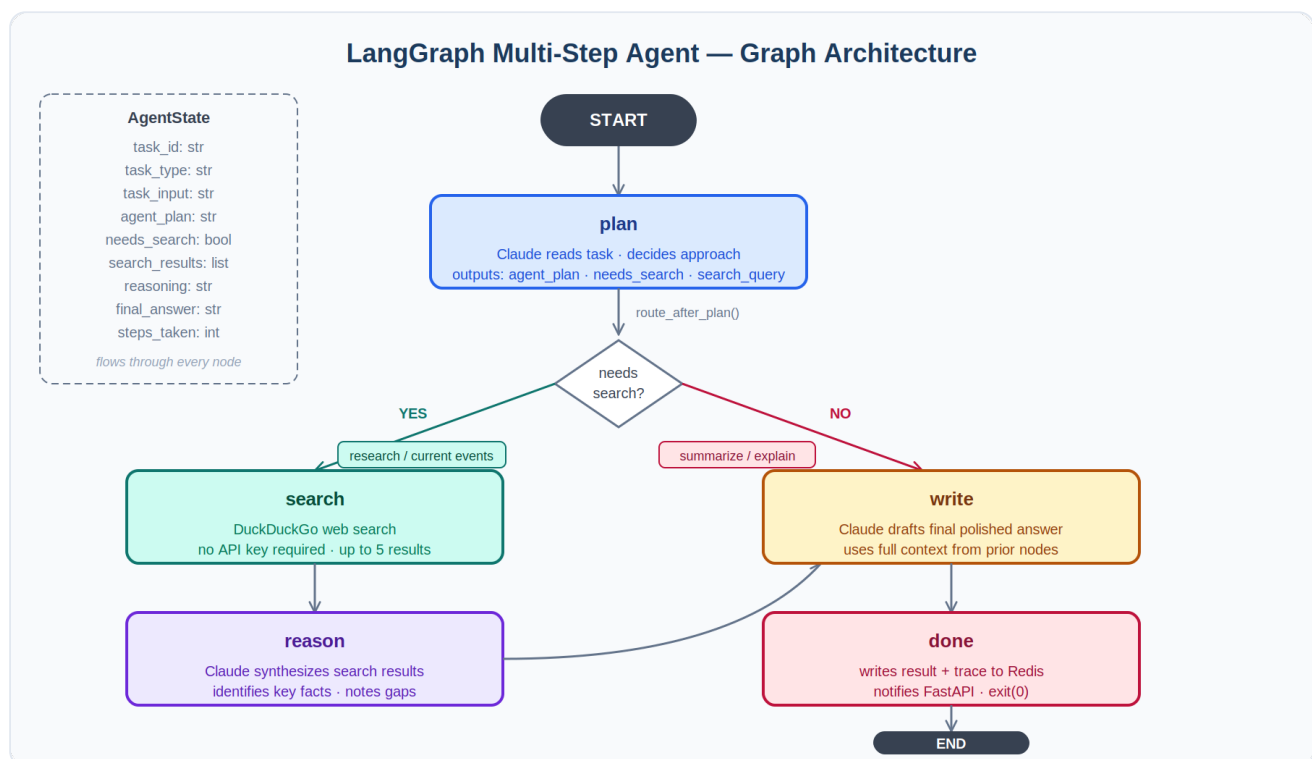


Figure 8 — LangGraph state machine. Claude makes conditional decisions at each node. The search path triggers on research tasks; the direct path is taken for summarization and explanation tasks.

AgentState — shared memory across all nodes

Every node receives the full state and returns a partial update. State flows through the graph accumulating context at each step.

```
class AgentState(TypedDict):
    task_id:      str      # K8s Job injects this via env var
    task_type:    str      # "research" | "summarize"
    task_input:   str      # the user's prompt
    agent_plan:   str      # Claude's plan – set by plan node
    needs_search: bool     # Claude's decision – set by plan node
    search_query: str      # what to search – set by plan node
    search_results: List[str] # DuckDuckGo results – set by search node
    reasoning:    str      # Claude's synthesis – set by reason node
    final_answer: str      # final response – set by write node
    steps_taken:  int      # incremented by every node
    error:        str      # set on exception
```

The five nodes

Node	What it does	Output to state
<code>plan</code>	Claude reads the task and decides the approach. Outputs a structured JSON plan including whether web search is needed and what to search for. Entry point of the graph.	<code>agent_plan</code> , <code>needs_search</code> , <code>search_query</code>
<code>search</code>	Runs a DuckDuckGo web search using the query from the plan node. No API key required. Collects up to 5 results. Falls back gracefully if rate-limited.	<code>search_results</code>
<code>reason</code>	Claude synthesizes the search results — identifies key facts, notes gaps and conflicts, and summarizes what was learned. Only runs on the search path.	<code>reasoning</code>
<code>write</code>	Claude drafts the final polished answer using all prior context: the plan, search results, and reasoning. System prompt adapts to task type.	<code>final_answer</code>
<code>done</code>	Writes the complete result and trace to Redis. Calls FastAPI to trigger metric recording. Returns <code>steps_taken</code> to satisfy LangGraph's state update requirement.	<code>steps_taken</code> (no-op update)

Conditional routing

After `plan`, a routing function reads `needs_search` from state and returns the name of the next node as a string. LangGraph uses this to decide which edge to follow.

```
def route_after_plan(state: AgentState) -> str:
    if state.get("needs_search"):
        return "search"    # → search → reason → write → done
    return "write"         # → write → done (faster path)
```

Research tasks about current events take the search path (4-5 steps). Explanation and summarization tasks skip search entirely (2-3 steps). Claude decides which path based on the task content — not hardcoded rules.

The result payload

The `done` node writes this structure to Redis. The full trace is available via `GET /tasks/{id}`.

```
{
  "answer": "Final polished response from Claude...",
  "trace": {
    "plan": "My approach: search for recent K8s releases...",
    "needed_search": true,
    "search_results": ["Kubernetes 1.30 released April 2024...", ...],
    "reasoning": "The search results confirm that 1.30 introduced...",
    "steps_taken": 4
  }
}
```

What changed in worker/requirements.txt

```
redis==5.0.4
anthropic==0.40.0
httpx==0.27.2
langgraph==0.1.19
langchain-anthropic==0.1.22
langchain-community==0.2.10
duckduckgo-search==5.3.1b1 # pinned to avoid Rust dependency
```

GOTCHAS LEARNED

Two errors hit during implementation. (1) `plan` is a reserved key in LangGraph state — rename conflicting keys with a prefix like `agent_plan`. (2) Every node must return at least one state key — returning `{}` from `done` raises `InvalidUpdateError`. Return a no-op update like `{"steps_taken": state["steps_taken"]}`.

SECTION 8 — HELM CHART

Packaging the full application

A Helm chart makes the entire system installable in one command on any Kubernetes cluster. The same chart that runs on k3d locally deploys identically to EKS, GKE, or AKS — the only thing that changes is the values file.

WHAT HELM ADDS

Without Helm you apply 10+ YAML files in a specific order, track which values need to be consistent across files, and manually clean up when uninstalling. With Helm: one install command, one upgrade command, one uninstall command. Values are the single source of truth. The chart validates required inputs and fails loudly if they're missing.

How Helm works

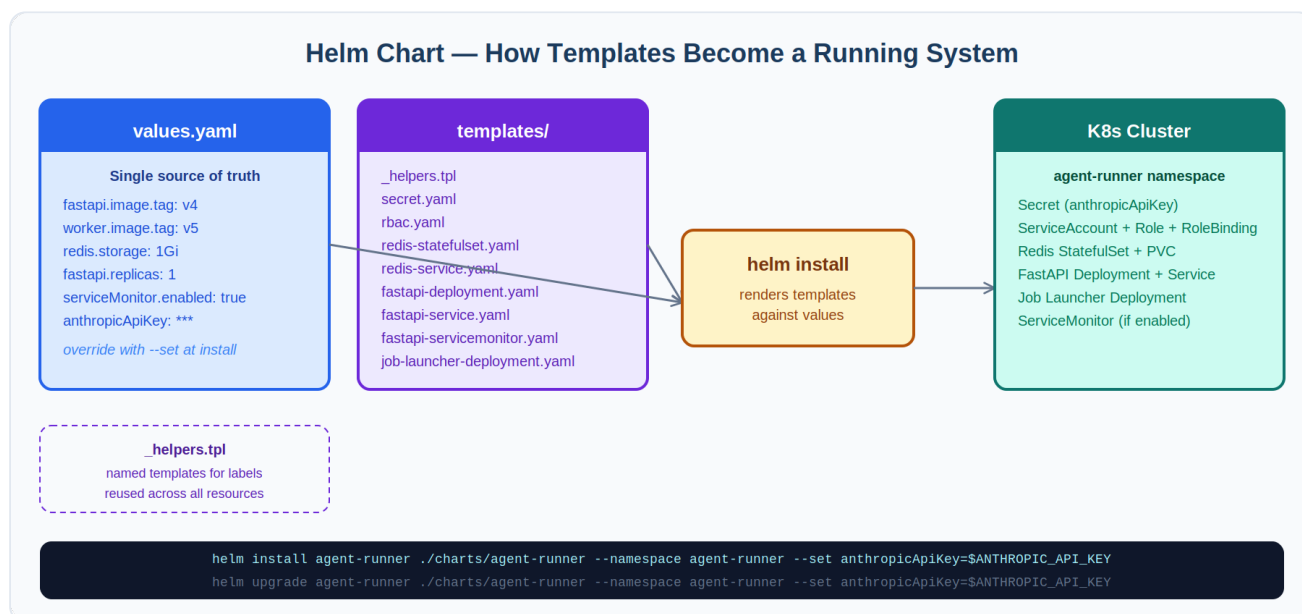


Figure 9 — Helm renders templates against values.yaml and applies the result to Kubernetes. The same chart, different values, different environments.

Chart structure

```
charts/agent-runner/  
├─ Chart.yaml           # identity: name, version, appVersion  
├─ values.yaml          # defaults for all configurable values  
└─ templates/  
    ├─ _helpers.tpl     # named templates – shared labels and selectors  
    ├─ secret.yaml      # ANTHROPIC_API_KEY – fails if not set  
    ├─ rbac.yaml        # ServiceAccount + Role + RoleBinding  
    ├─ redis-statefulset.yaml  
    ├─ redis-service.yaml  
    ├─ fastapi-deployment.yaml  
    ├─ fastapi-service.yaml  
    ├─ fastapi-servicemonitor.yaml # conditional on serviceMonitor.enabled  
    └─ job-launcher-deployment.yaml
```

values.yaml — the single source of truth

```
namespace: agent-runner  
  
fastapi:  
  image:  
    repository: agent-api  
    tag: "v4"  
    pullPolicy: Never  
  replicas: 1  
  port: 8000  
  
worker:  
  image:  
    repository: agent-worker  
    tag: "v5"  
    pullPolicy: Never  
  job:  
    backoffLimit: 2  
    activeDeadlineSeconds: 300  
    ttlSecondsAfterFinished: 600  
  
redis:  
  storage: "1Gi"  
  
anthropicApiKey: "" # set via --set at install time – never hardcode here  
  
serviceMonitor:  
  enabled: true # set false if Prometheus is not installed
```

[_helpers.tpl](#) — why it exists

Named templates enforce consistency. Every resource uses the same label set and selector pattern, defined once and included everywhere. This prevents the subtle label mismatches that break Service-to-Pod routing and Prometheus ServiceMonitor discovery.

```
{/* Common labels applied to every resource */}
{- define "agent-runner.labels" -}
helm.sh/chart: {{ .Chart.Name }}-{{ .Chart.Version }}
app.kubernetes.io/name: {{ include "agent-runner.name" . }}
app.kubernetes.io/instance: {{ .Release.Name }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
{- end -}

{/* Selector labels — stable across upgrades */}
{- define "agent-runner.selectorLabels" -}
app.kubernetes.io/name: {{ include "agent-runner.name" . }}
app.kubernetes.io/instance: {{ .Release.Name }}
{- end -}
```

LABELS VS SELECTORS

Common labels include the chart version — they change on every upgrade. Selector labels must be immutable — Kubernetes will not let you change a Deployment's selector after creation. Keep selectors minimal and stable. This is why they are defined separately in [_helpers.tpl](#) rather than reusing the full label set.

Fail-fast validation in `secret.yaml`

```
{- if .Values.anthropicApiKey }
apiVersion: v1
kind: Secret
metadata:
  name: agent-secrets
  namespace: {{ .Values.namespace }}
type: Opaque
stringData:
  ANTHROPIC_API_KEY: {{ .Values.anthropicApiKey | quote }}
{- else }
{- fail "anthropicApiKey must be set. Use --set anthropicApiKey=<your-key>" }
{- end -}
```

The `{- fail ... }` directive makes Helm refuse to install with a helpful error message if the API key is not provided. This is better than silently creating a broken deployment that fails at runtime.

Conditional rendering — `serviceMonitor.enabled`

```
{- if .Values.serviceMonitor.enabled }  
apiVersion: monitoring.coreos.com/v1  
kind: ServiceMonitor  
  ...  
{- end }
```

The entire ServiceMonitor block disappears from the rendered output when `serviceMonitor.enabled: false`. This means the chart installs cleanly on clusters that don't have Prometheus — you don't get CRD-not-found errors for resources that shouldn't exist.

Install, upgrade, uninstall

```
# Validate templates before touching the cluster  
helm template agent-runner ./charts/agent-runner --set anthropicApiKey=test  
helm lint ./charts/agent-runner --set anthropicApiKey=test  
  
# Install into a dedicated namespace  
helm install agent-runner ./charts/agent-runner \  
  --namespace agent-runner \  
  --create-namespace \  
  --set anthropicApiKey=$ANTHROPIC_API_KEY  
  
# Upgrade after any change to templates or values  
helm upgrade agent-runner ./charts/agent-runner \  
  --namespace agent-runner \  
  --set anthropicApiKey=$ANTHROPIC_API_KEY  
  
# Inspect what is running  
helm get values agent-runner -n agent-runner  
helm get manifest agent-runner -n agent-runner  
  
# Clean uninstall – removes all chart resources  
helm uninstall agent-runner -n agent-runner
```

PRODUCTION TIP

In CI/CD pipelines use `helm upgrade --install` instead of separate install/upgrade commands. `helm upgrade --install` is idempotent — it installs if the release does not exist and upgrades if it does. This means your pipeline works correctly on first deploy and on every subsequent deploy without branching logic.